

Multi-Camera Vision System for Automated Material Detection and Sorting

Project Overview:

A real-time computer vision system for enhancing high-value material recovery and worker safety monitoring on industrial conveyor belts. This system combines YOLOv5 for precise material detection and segmentation with intelligent background subtraction for motion analysis. The architecture features camera-specific region-of-interest (ROI) processing, worker-interaction filtering to minimize false positives, and a robust counting mechanism. The system achieved 96.8% mAP@[0.5:0.95] for material detection and 74.5% mAP@[0.5:0.95] for worker detection, with <15ms inference latency, enabling real-time monitoring across multiple camera feeds.

GitHub Repository: [CVAnnotate](#)

Key Technologies and Skills Used:

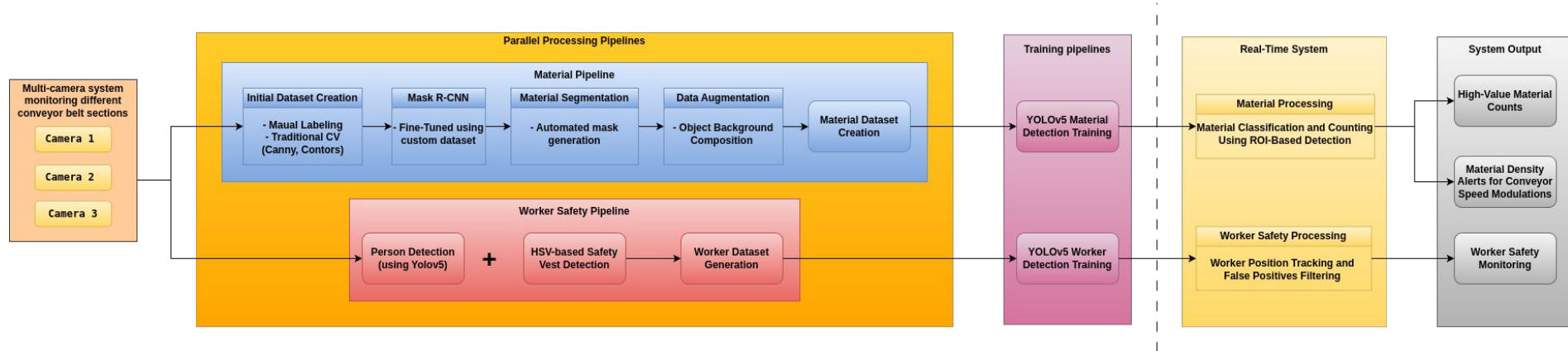
Languages: Python, C++

Frameworks: PyTorch, OpenCV, scikit-learn, NumPy, Pandas

Machine Learning: Model Training, Fine-tuning, Data Augmentation, Model Evaluation (mAP, Precision, Recall)

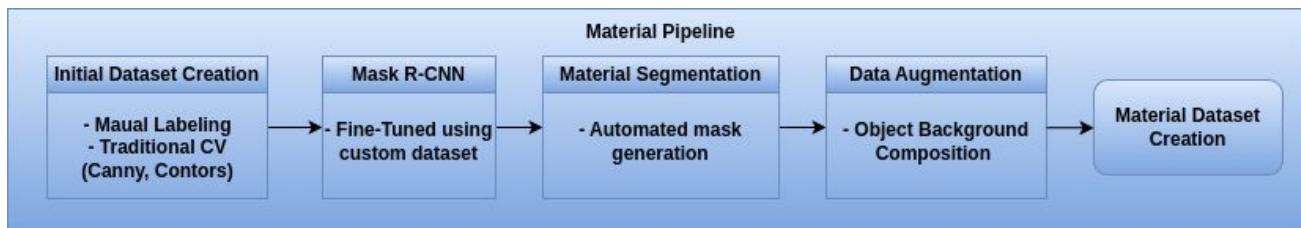
Computer Vision: Object Detection (YOLOv5), Instance Segmentation (Mask R-CNN), Multi-Camera Systems, Region of Interest (ROI) Processing, Background Subtraction (MOG2), Image Processing

Pipeline



End-to-end pipeline for automated material sorting and worker safety, with parallel processing for offline training and real-time operation

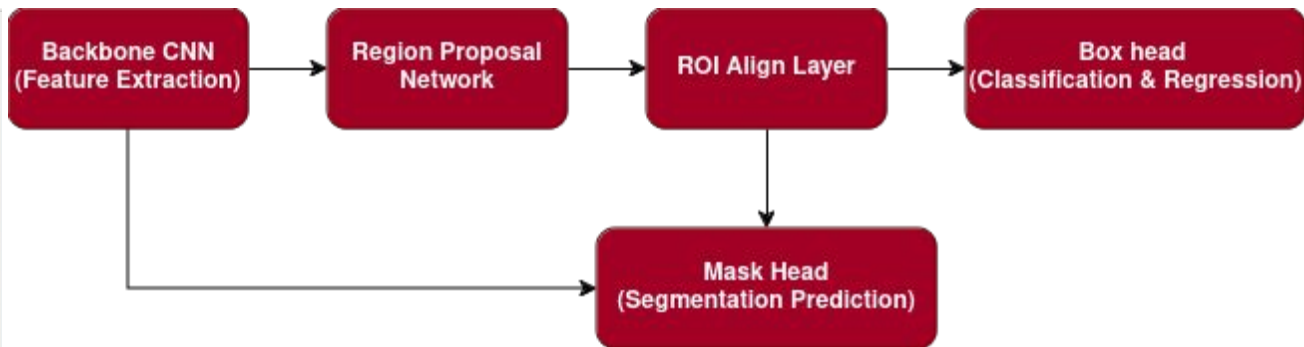
Semi-Automated Data Annotation and Augmentation



High-Level System view



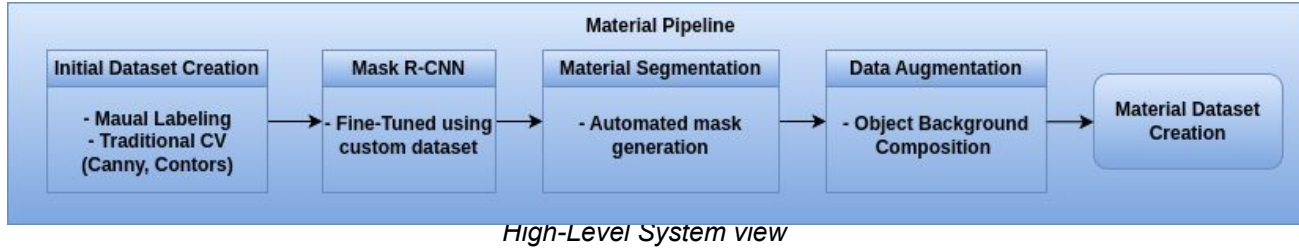
Initial Dataset Creation using labelme



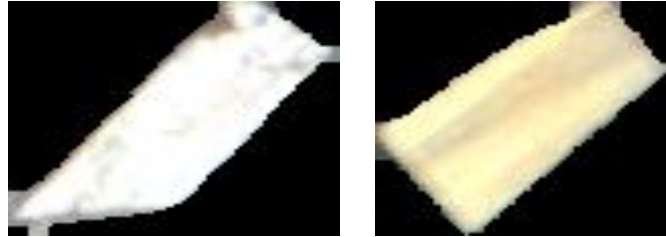
Simple Mask R-CNN Architecture Diagram

- 1. Initial Dataset:** A small initial dataset of ~800 images (200 per material class) was created using a combination of manual annotation (with LabelMe) and traditional computer vision techniques (Canny edge detection, adaptive thresholding) to generate initial segmentation masks.
- 2. Mask R-CNN Fine-tuning:** A pre-trained Mask R-CNN model was fine-tuned on this initial dataset, significantly improving segmentation accuracy.

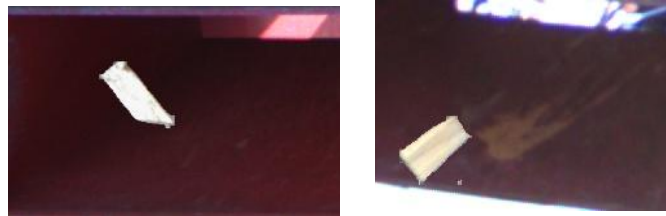
Data Augmentation and Dataset Creation



Initial Dataset Creation using labelme



Automated Masked & Segmented images

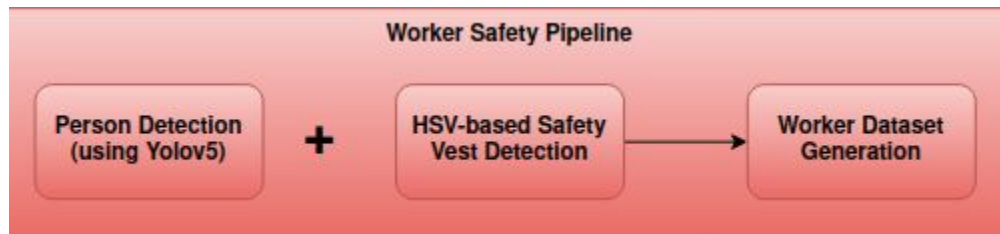


Data Augmentation (Object-Background Compositing)

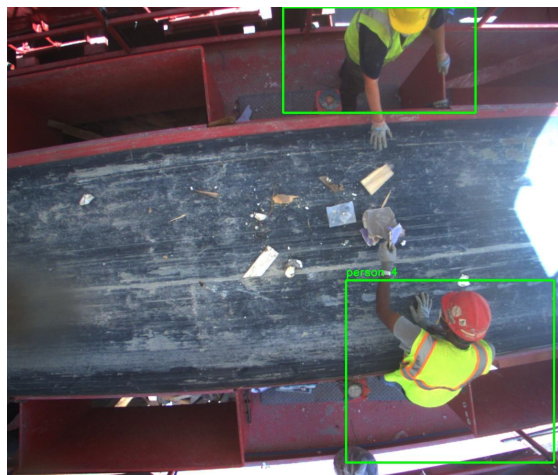
Automated Segmentation: The fine-tuned Mask R-CNN was then used to automatically generate segmentation masks for a much larger dataset of 43,000+ images captured from the live camera feeds. This drastically reduced manual annotation effort.

Data Augmentation: To create a robust and diverse training dataset, a sophisticated data augmentation strategy was employed. Segmented material images (generated by the Mask R-CNN model) were overlaid onto images of the empty conveyor belt bins

Worker Detection Dataset Creation



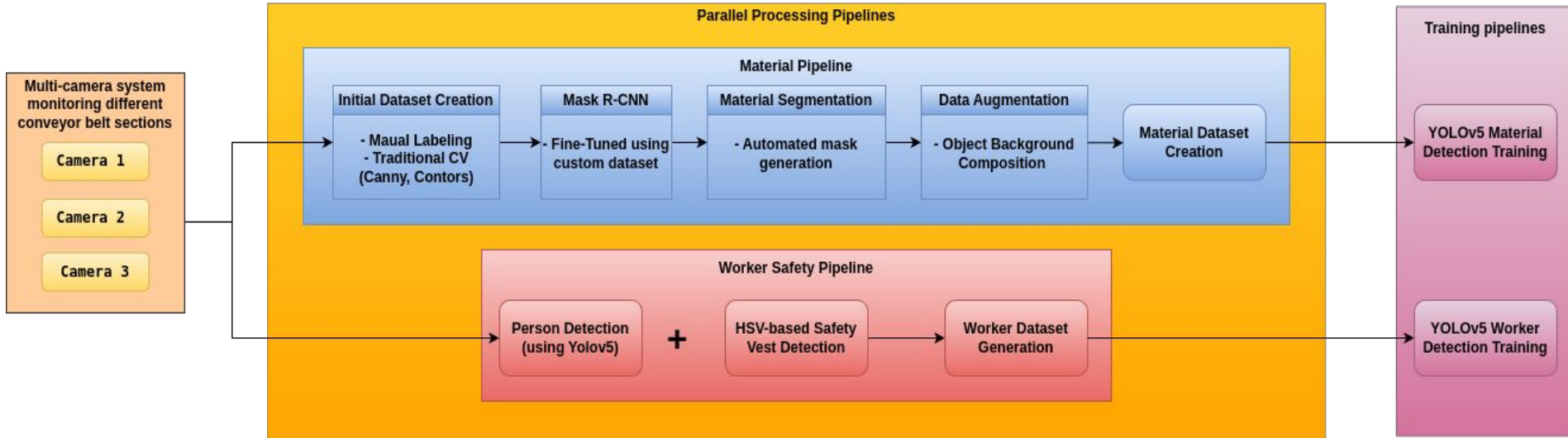
Raw image from the camera



Worker detection using a combination of HSV-based color thresholding (for initial detection) and a trained YOLOv5 model (for refined bounding box prediction)

A separate dataset was created for training the worker detection model. To quickly gather initial bounding box annotations, a color-based detection method was used, targeting the bright yellow safety vests commonly worn by workers. This initial detection was then used to train a YOLOv5 model for more robust person detection.

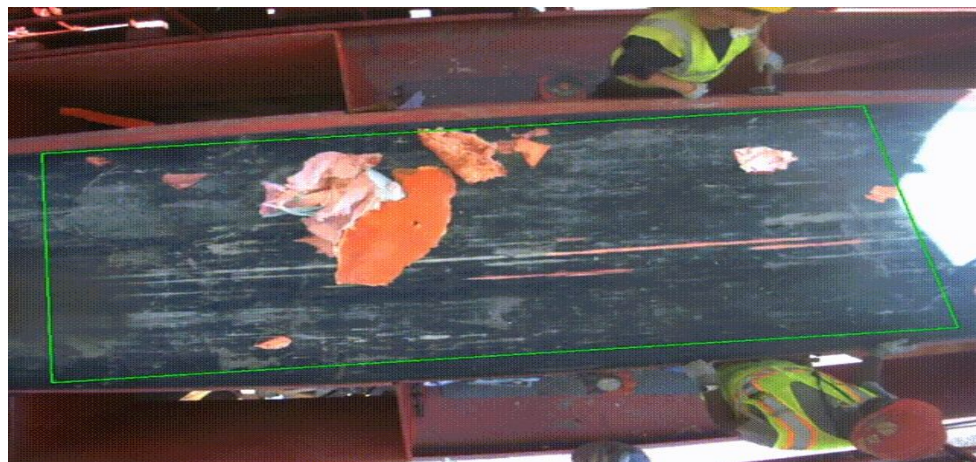
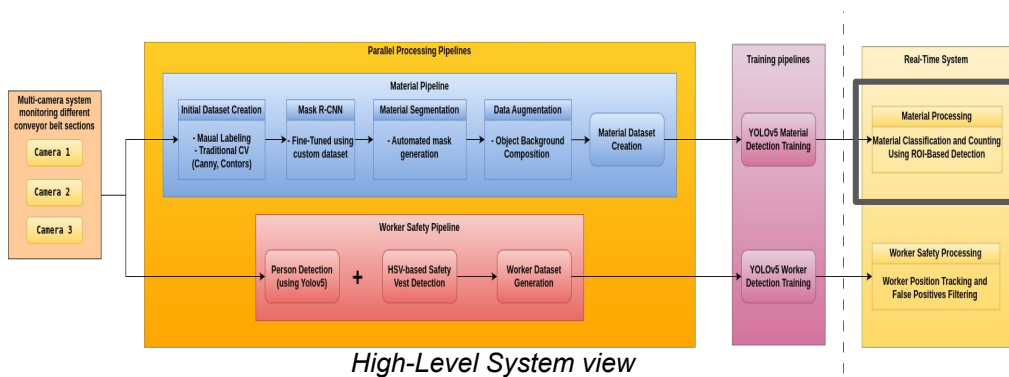
Detection Model Training and Performance



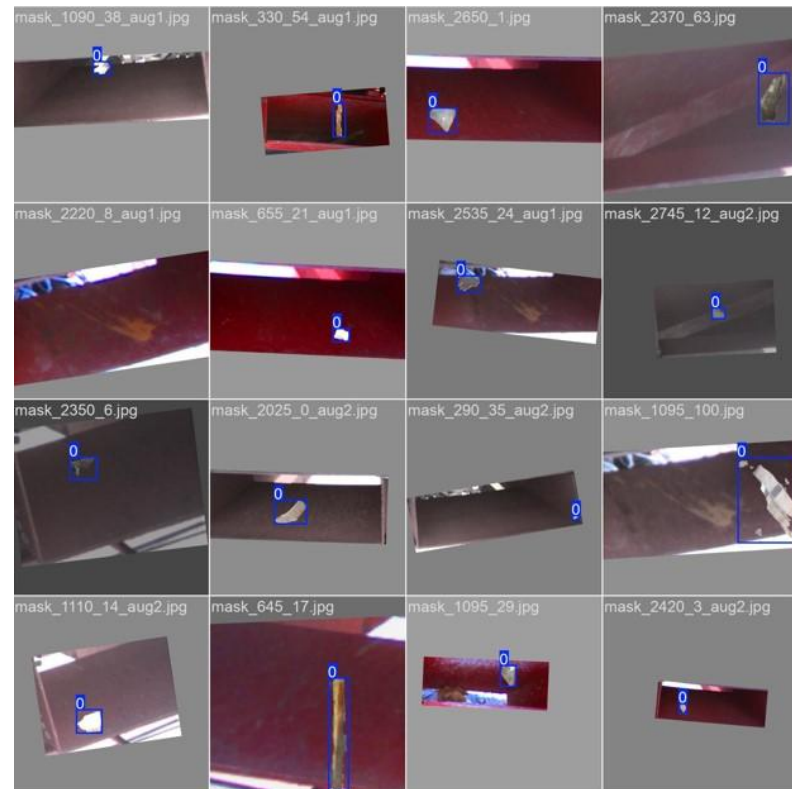
Performance:

- Material Detection Model: 96.8% mAP@[0.5:0.95]
- Worker Detection Model: 74.5% mAP@[0.5:0.95]

Real-Time Processing System

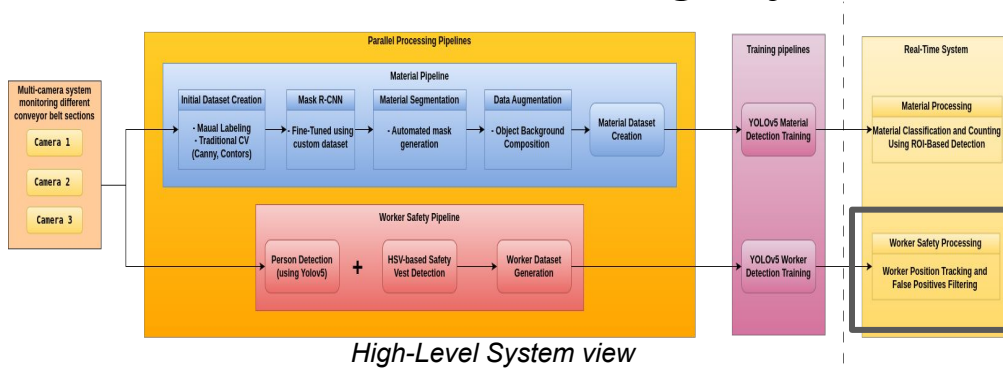


Real-time material detection and counting. Green boxes indicate detected and counted materials within the defined ROI



Real-time detection and counting results from the trained model

Real-Time Processing System



Real Time worker position tracking

False Positive Filtering:

- **Worker Overlap:** Detected materials overlapping with detected workers are *not* counted. This prevents miscounting worker interactions as materials.
- **Cooldown:** After counting an object in an ROI, a short cooldown prevents immediately recounting the *same* object.



Examples of False Positives